



Marwadi
University
Marwadi Chandarana Group

NAAC



MARWADI UNIVERSITY

FACULTY OF COMPUTER APPLICATIONS

Programming with Python & Problem Solving (05MC0107)

LAB MANUAL

Programming with Python & Problem Solving (05MC0107) Lab Manual (2026-27)

Program: MCA

Semester: 1

Name of the Student: Kundariya Vasu Rameshbhai

Enrolment No. :92600584063

I

NDEX

Lab	Program(unit 1)	Date	Remarks	Signature
1	Write a program to demonstrate basic input and output operations in Python.			
2	Write a program to illustrate the use of different data types and type casting.			
3	Write a program to perform arithmetic relational and logical operations using Python operators.			
4	Write a program to demonstrate string operations including slicing formatting and built-in string functions.			
5	Write a program to create and manipulate lists using indexing slicing and list comprehensions.			
6	Write a program to illustrate the use of tuples and sets with basic operations.			
7	Write a program to create a dictionary and demonstrate dictionary methods and iteration.			
8	Write a program to explain mutable and immutable objects in Python.			
9	Write a program to define and use user-defined functions with different types of arguments.			
10	Write a program to demonstrate recursion using factorial or Fibonacci series.			

Practical No. 1

Definition : Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:

- To understand how to accept input from the user in Python.
- To display output using the print() function.
- To perform basic operations on user-provided data.
- To understand basic Python syntax for input and output.

Source Code:

```
name=input("Enter your name :=")
age = int(input("Enter your age :="))
print("hello",name)
print("your age is ",age)
```

Output:

```
Enter your name :=vasu
Enter your age :=21
hello vasu
your age is 21
```

Conclusion:

The program successfully demonstrates the basic input and output operations in Python. It shows how to accept data from the user using input(), convert input into appropriate data types, store values in variables, perform a basic arithmetic operation, and display the results using print()

Practical No. 2

Definition .: Write a program to illustrate the use of different data types and type casting.

Aim/ Objective:

- To understand different data types in Python.
- To learn how to use integers, floating-point numbers, strings, and Boolean values.
- To understand type casting in Python.
- To perform conversions between different data types.
- To display the results using the print() function.

Source Code:

```
name = "vasu"

age = 21

hight = 5.6

is_student = True

print("name :",name, "Type :", type(name))

print("age :",age, "Type :", type(age))

print("hight :",hight, "Type :", type(hight))

print("is__student :",is_student, "Type :", type(is_student))

num1="25"

num2=int(num1)

num3=float(num2)

num4=str(num2)

print("string to integer :-",num2)

print("integer to float :-",num3)

print("integer to string :-",num4)
```

Output:

```
name :, vasu Type : <class 'str'>
age :, 21 Type : <class 'int'>
hight :, 5.6 Type : <class 'float'>
is__student :, True Type : <class 'bool'>

string to integer :- 25

integer to float :- 25.0

integer to string :- 25
```

Conclusion:

The program successfully demonstrates the use of different data types in Python. It shows how to store values using integers, floating-point numbers, strings, and Boolean data types. It also demonstrates type casting by converting one data type into another and displaying the results using the print() function.

Practical No. 3

Definition : Write a program to perform arithmetic relational and logical operations using Python operators.

Aim/ Objective:

- To understand different types of operators in Python.
- To perform arithmetic operations such as addition, subtraction, multiplication, and division.
- To perform relational operations to compare two values.
- To perform logical operations using and, or, and not.
- To understand how Python operators are used in programs.

Source Code:

```
a=5

b=10

print("a=",a)

print("b=",b)

print("-----Arithmetic operation-----")

print("Addition =",a+b)

print("Subraction =",a-b)

print("Maltipiction =",a*b)

print("Division=",a/b)

print("modulus =",a%b)

print("Power=",a**b)

print("Floor division=",a//b)

print("-----Relational operation-----")

print("a==b ",a==b)
```

```
print("a!= ",a!=b)
print("a<b ",a<b)
print("a>b ",a>b)
print("a<=b ",a<=b)
print("a>=b ",a>=b)
print("-----logical operation-----")
x = True
y = False
print("x and y= ",x and y)
print("x or y= ",x or y)
print("not x = ", not x)
```

Output:

```
a= 5
b= 10
-----Arithmetic operation-----
Addition = 15
Subtraction = -5
Maltipiction = 50
Division= 0.5
modulus = 5
Power= 9765625
Floor division= 0
-----Relational operation-----
a==b False
```

a!= True

a<b True

a>b False

a<=b True

a>=b False

-----logical operation-----

x and y= False

x or y= True

not x = False

Conclusion:

The program successfully demonstrates arithmetic, relational, and logical operations using Python operators. It shows how to perform basic mathematical calculations, compare values, and use logical conditions. This helps in understanding how different Python operators are used to perform calculations and make decisions in a program.

Practical No. 4

Definition : Write a program to demonstrate string operations including slicing formatting and built-in string functions.

Aim/ Objective:

- To understand the concept of strings in Python.
- To perform string slicing operations.
- To learn string formatting in Python.
- To use built-in string functions such as upper(), lower(), replace(), and len().
- To display the results of different string operations using the print() function.

Source Code:

```
Text = "My name is vasu"

print("Original string =", Text)

print("length = ",len(Text))

print("Addition ",Text+" From Rajkot")

print("Ripition ", Text*2)

print("-----Slicing-----")

print("First 5 character =", Text[:5])

print("Last 5 character =", Text[6:])

print("character 1 to 7 =", Text[1:7])

print("Revese ",Text[::-1])

print("-----Formatting-----")

name="vasu"

age=21

print("My name is {} and I am {} years old.".format(name,age))

print(f'My name is {name} I am {age} year old.')

print("----- built-in string functions-----")
```

```
print("Uppercase =",Text.upper())  
print("Lowercase =",Text.lower())  
print("Replace =",Text.replace("vasu","Meet"))  
print("Count",Text.count("o"))  
print("find =",Text.find("vasu"))
```

Output:

```
original string = My name is vasu  
length = 15  
Addition My name is vasu From Rajkot  
Ripition My name is vasuMy name is vasu  
-----Slicing-----  
First 5 character = My na  
Last 5 character = e is vasu  
character 1 to 7 = y name  
Revese usav si eman yM  
-----Formatting-----  
My name is vasu and I am 21 years old.  
My name is vasu I am 21 year old.  
----- built-in string functions-----  
Uppercase = MY NAME IS VASU  
Lowercase = my name is vasu  
Replace = My name is Meet  
Count 0  
find = 11
```

Conclusion:

The program successfully demonstrates different string operations in Python. It shows how to perform string slicing, format strings, and use built-in string functions. This helps in understanding how strings can be created, modified, and processed easily using Python.

Practical No. 5

Definition : Write a program to create and manipulate lists using indexing slicing and list comprehensions.

Aim/ Objective:

- To understand the concept of lists in Python.
- To create and store multiple values in a list.
- To access list elements using indexing.
- To perform slicing operations on lists.
- To use list comprehensions to create a new list.
- To understand how to manipulate list elements in Python.
-

Source Code:

```
num = [1,2,3,4,5,6,7,8,9,10]

print(num)

print("First element :",num[0])

print("Secend element :",num[1])

print("last element :",num[-1])


print("-----Slicing-----")

print("First Three element :",num[:3])

print("Last three element :",num[6:])

print("three to seven element:",num[2:6])

num.append(11)


print("After append: ", num)

num.remove(10)
```

```
print("After remove :", num)

squares = [x * x for x in num]

print("Squares :",squares)

even_number = [x for x in num if x %2==0]

print("Even Number : ",even_number)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

First element : 1

Secend element : 2

last element : 10

-----Slicing-----

First Three element : [1, 2, 3]

Last three element : [7, 8, 9, 10]

three to seven element: [3, 4, 5, 6]

After append: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

After remove : [1, 2, 3, 4, 5, 6, 7, 8, 9, 11]

Squares : [1, 4, 9, 16, 25, 36, 49, 64, 81, 121]

Even Number : [2, 4, 6, 8]
```

Conclusion:

The program successfully demonstrates how to create and manipulate lists in Python. It shows how to access elements using indexing, extract elements using slicing, and create new lists using list comprehensions. This helps in understanding the basic operations and flexibility of lists in Python.

Practical No. 6

Definition : Write a program to illustrate the use of tuple

Aim/ Objective:

- To understand the concept of tuples in Python.
- To create and store multiple values in a tuple.
- To access tuple elements using indexing.
- To perform tuple slicing operations.
- To use built-in tuple functions such as len(), count(), and index().

Source Code:

```
num = (10,20,30,40,50,60,70,80,90,100)

print(num)

print("Frist element :",num[0])

print("Last element :",num[-1])

print("four to nine element :",num[3:9])

print("Lenth of Tuple : " ,len(num))

print("Count of : ",num.count(20))

print("index of 30 :",num.index(50))
```

Output:

```
(10, 20, 30, 40, 50, 60, 70, 80, 90, 100)

Frist element : 10

Last element : 100

four to nine element : (40, 50, 60, 70, 80, 90)

Lenth of Tuple : 10

Count of :  1

index of 30 : 4
```

Conclusion:

The program successfully demonstrates the use of tuples in Python. It shows how to create a tuple, access its elements using indexing, perform slicing, and use built-in tuple functions. This helps in understanding how tuples are used to store multiple values in a fixed and ordered collection.

Definition :Write a program to create a dictionary and demonstrate dictionary methods and iteration.

Aim/ Objective:

- To understand the concept of dictionaries in Python.
- To create a dictionary using key-value pairs.
- To access and modify dictionary elements.
- To demonstrate built-in dictionary methods such as keys(), values(), items(), and get().
- To iterate through a dictionary using a for loop

Source Code:

```
print("-----Dictionary-----")

student = {

    "name":"vasu",

    "age":21,

    "course":"Python"

}

print(student)

print("Name",student["name"])

print("Age",student["age"])

print("Course",student["course"])


print("-----dictionary method-----")

print("Keys :",student.keys())

print("Values :",student.values())

print("Item :",student.items())

student["city"] = "Rajkot"
```

```
print("After addind city :",student)

student["age"] = "22"

print("After Upadating Age:",student)

student.pop("city")

print("After removing city",student)

for key,value in student.items():

    print(key,":",value)
```

Output:

-----Dictionary-----

```
{'name': 'vasu', 'age': 21, 'course': 'Python'}
```

Name vasu

Age 21

Course Python

-----dictionary method-----

```
Keys : dict_keys(['name', 'age', 'course'])
```

```
Values : dict_values(['vasu', 21, 'Python'])
```

```
Item : dict_items([('name', 'vasu'), ('age', 21), ('course', 'Python')])
```

```
After addind city : {'name': 'vasu', 'age': 21, 'course': 'Python', 'city': 'Rajkot'}
```

```
After Upadating Age: {'name': 'vasu', 'age': '22', 'course': 'Python', 'city': 'Rajkot'}
```

```
After removing city {'name': 'vasu', 'age': '22', 'course': 'Python'}
```

```
name : vasu
```

```
age : 22
```

```
course : Python
```

Conclusion:

The program successfully demonstrates how to create and manipulate a dictionary in Python. It shows how to access key-value pairs, use different dictionary methods, and iterate through the dictionary using a loop. This helps in understanding how dictionaries are used to store and manage data in key-value pair format.

Definition : 8. Write a program to explain mutable and immutable objects in Python.Immutable object

Aim/ Objective:

- To understand the concept of mutable and immutable objects in Python.
- To learn how immutable objects cannot be changed after creation.
- To understand examples of immutable objects such as integers, strings, and tuples.
- To observe how assigning a new value creates a new object instead of changing the original object.

Source Code:

```
a =(10,20,30,40,50,)
print(a)

name = "Vasu"
print(name)

name = name+" Patel"
print("After changing string",name)


# Mutable object
num =[10,20,30,40,50]
print(num)
num[0]=100
print("After changing list",num)
num.append(40)
print("After append list",num)
```

Output:

(10, 20, 30, 40, 50)

Vasu

After changing string Vasu Patel

[10, 20, 30, 40, 50]

After changing list [100, 20, 30, 40, 50]

After append list [100, 20, 30, 40, 50, 40]

Conclusion:

The program successfully demonstrates the concept of mutable and immutable objects in Python. It shows that immutable objects, such as integers, strings, and tuples, cannot be changed after they are created. Any modification creates a new object with a new value. This helps in understanding the difference between mutable and immutable objects in Python.

Definition : Write a program to define and use user-defined functions with different types of arguments.

Aim/ Objective:

- To understand the concept of user-defined functions in Python.
- To learn how to define and call functions.
- To use different types of arguments in functions.
- To understand positional, keyword, default, and variable-length arguments.
- To display the results returned by user-defined functions.
-

Source Code:

```
def add(a,b):  
    print("Addition :",a+b)  
add(10,20)  
  
def student(name,age):  
    print("name :",name)  
    print("age :",age)  
student(name ="vasu",age=21)  
  
def greet(name="vasu"):  
    print("hello",name)  
greet()  
  
def total(*number):  
    print("total :",sum(number))  
total(10,20,30,40)
```

Output:

Addition : 30

name : vasu

age : 21

hello vasu

total : 100

Conclusion:

The program successfully demonstrates how to define and use user-defined functions in Python. It shows the use of different types of arguments, including positional, keyword, default, and variable-length arguments. This helps in understanding how functions make Python programs easier to organize, reuse, and manage.

Practical No. 10

Definition : Write a program to demonstrate recursion using factorial or Fibonacci series.

Aim/ Objective:

- To understand the concept of recursion in Python.
- To learn how a function can call itself.
- To demonstrate recursion using the factorial of a number.
- To understand the base condition and recursive condition.
- To display the result using the print() function.

Source Code:

```
def factorial(n):  
    if n==0 or n==1:  
        return 1  
    else :  
        return n * factorial (n-1)  
  
num =5  
  
result = factorial(num)  
  
print("factorial of",num,"is :",result)
```

Output:

factorial of 5 is : 120

Conclusion:

The program successfully demonstrates the concept of recursion in Python using the factorial of a number. It shows how a function calls itself repeatedly until the base condition is reached. This helps in understanding how recursive functions work and how they can be used to solve problems step by step.